

Hotel Manager

Relatório do Sistema de Gerenciamento

Computação I – UFRJ – 2025/1

Denis Nicolau – 126031761

1. Visão Geral

Um sistema de gerenciamento de hotel completo, executado inteiramente no terminal. O programa foi desenvolvido em Python

1.1 Módulos Importados

O programa utiliza três módulos da biblioteca padrão do Python:

- **os** — verifica se os arquivos .txt já existem no disco antes de tentar abri-los (`os.path.exists`).
- **random** — gera o código único de 4 dígitos de cada reserva via `random.randint(1000, 9999)`.
- **datetime / timedelta** — registra data e hora do check-in e checkout, e calcula o intervalo de tempo para as estatísticas (últimos 30 ou 365 dias).

1.2 Estrutura de Dados Principal

O programa usa três estruturas compostas que ficam em memória durante a execução:

usuarios — Dicionário de dicionários

```
{ "denis": { "senha": "123", "email": "d@d.com", "area": "administracao",  
  "is_admin": True } }
```

A chave externa é o nome do usuário. O valor é outro dicionário com seus dados. Isso permite busca direta por nome em tempo constante.

reservas — Dicionário de dicionários (com listas internas)

```
{ "7342": { "codigo": "7342", "cpfs": ["123.456.789-00"], "nomes": ["João"],  
  "noites": 3, "suite_id": 1, "quarto": 101,  
  "valor_diarias": 1200.0, "gastos_restaurante": 85.0,  
  "pedidos": [ { "data": "...", "itens": [("Frango", 35.0)], "subtotal":  
35.0 } ],  
  "ativa": True, "checkin": "...", "checkout_dt": "" } }
```

Cada reserva é um dicionário com vários tipos de dados: strings, floats, booleanos e uma lista de pedidos. Os pedidos por sua vez são dicionários com uma lista de tuplas (nome, preço).

CARDAPIO — Dicionário de listas de tuplas

```
{ "Comidas": [ ("Frango Grelhado", 35.0), ("Filé", 55.0), ... ],  
  "Bebidas": [ ("Água Mineral", 6.0), ... ],  
  "Sobremesas": [ ("Petit Gâteau", 22.0), ... ] }
```

As categorias são as chaves, e os itens são listas de tuplas (nome, preço). Tuplas foram usadas porque cada item é imutável — nome e preço vêm sempre juntos.

2. Explicação Blocos

2.1 Bloco de Persistência (salvar / carregar)

Este bloco é responsável por toda comunicação com os arquivos .txt. É composto por quatro funções:

salvar_usuarios()

Abre usuarios.txt no modo escrita ('w') e percorre o dicionário usuarios. Cada usuário vira uma linha no formato:

```
nome|senha|email|area|is_admin
```

O separador | foi escolhido porque não aparece em nenhum dos dados do usuário. O arquivo é sobrescrito completamente a cada chamada.

salvar_reservas()

Cada reserva ocupa um bloco de linhas separado por '---'. A estrutura dentro do bloco é:

```
---
codigo|ativa|qtd_pessoas|noites|suite_id|suite_nome|quarto
valor_diarias|gastos_restaurante|checkin|checkin_dt
checkout|checkout_dt|total_pago    ← ou 'sem_checkout'
cpf1,cpf2,cpf3
nome1,nome2,nome3
PEDIDOS|quantidade
data_pedido|subtotal|item1:preco;item2:preco
```

O campo 'sem_checkout' é um marcador para distinguir reservas ativas (sem checkout) de reservas encerradas.

salvar_cardapio()

Percorre o dicionário CARDAPIO e salva cada item em uma linha:

```
categoria|nome_item|preco
```

carregar_dados()

Chamada uma única vez ao iniciar o programa (em main()). Lê os três arquivos se existirem e reconstrói as variáveis em memória. Usa readline() em loop while para ler linha por linha, e split('|') para separar os campos.

Para as reservas, detecta o separador '---' e lê as próximas linhas em sequência, montando o dicionário da reserva campo a campo.

2.2 Bloco de Utilitários

Funções pequenas reutilizadas em todo o programa:

- **limpar()** — imprime o código ANSI `\033[2J\033[H` que limpa o terminal e volta o cursor ao topo.
 - **cabecalho(titulo)** — chama `limpar()` e imprime o cabeçalho.
 - **pausar()** — segura a tela com `input()` aguardando Enter, para o usuário ler o resultado antes de voltar ao menu.
 - **gerar_codigo(dict_reservas)** — gera um número aleatório de 4 dígitos e verifica se já existe no dicionário. Repete até encontrar um código livre. Recebe o dicionário como argumento e retorna o código.
 - **calcular_valor_diaria(suite, qtd, noites)** — recebe o dicionário da suíte, a quantidade de pessoas e o número de noites. Retorna o valor total: $(diaria + extra_pessoa * qtd) * noites$.
 - **buscar_quarto_disponivel(...)** — percorre as reservas ativas e monta um set de quartos ocupados. Depois testa os números de 101 a 115 (ou 201-215, 301-315) e retorna o primeiro livre. Retorna None se não houver vaga.
 - **tem_admin(dict_usuarios)** — percorre o dicionário de usuários e retorna True assim que encontrar um com `is_admin=True`. Usada na inicialização.
 - **calcular_totais(dict_reservas, limite)** — filtra apenas reservas com checkout realizado (`ativa=False` e `checkout_dt` preenchido), cujo `checkout_dt` seja posterior ao limite de data informado. Retorna uma tupla (`total_diarias`, `total_restaurante`).
 - **barra(pct, largura)** — recebe uma porcentagem e retorna uma string com blocos `█` e `░` proporcional ao valor. Usada nas estatísticas.
-

2.3 Bloco de Autenticação

setup_inicial()

Executada apenas quando não há nenhum administrador no sistema (primeira execução ou arquivo deletado). Solicita nome, email e senha, cria o usuário com `is_admin=True` e salva imediatamente.

tela_inicial()

Loop principal da tela de entrada. Exibe o menu Login / Registrar-se / Sair. Retorna o dicionário do usuário logado quando o login é bem-sucedido. Só sai do loop via `return` (login ok) ou `exit()` (opção 0).

fazer_login(dict_usuarios)

Recebe o dicionário de usuários como argumento. Lê nome e senha, busca o nome no dicionário com `.get()`, e compara a senha. Retorna o dicionário do usuário mesclado com o nome (usando `**`) ou `None` se falhar.

```
return {"nome": nome, **u} # expande o dicionário u e adiciona a chave "nome"
```

registrar_usuario()

Coleta nome, email, senha e área. Verifica se o nome já existe no dicionário antes de cadastrar. Salva imediatamente após o cadastro.

2.4 Bloco do Restaurante

registrar_pedido()

Fluxo em três etapas: (1) o garçom escolhe a categoria, (2) escolhe os itens pelo número (pode selecionar múltiplos digitando '1 3'), (3) informa o código da reserva. O pedido é somado ao campo `gastos_restaurante` da reserva e adicionado à lista pedidos.

A seleção múltipla funciona assim: a entrada '1 3' é transformada em lista ['1','3'] pelo `.split()`. Para cada elemento, tenta converter para `int` e buscar o item na lista. Itens inválidos são ignorados silenciosamente com `pass`.

listar_cardapio()

Percorre o dicionário `CARDAPIO` com `for cat, itens in CARDAPIO.items()`. Para cada categoria imprime um cabeçalho e lista os itens com `enumerate` para numerar.

2.5 Bloco de Administração

fazer_checkin()

Coleta os dados em etapas validando cada entrada com `try/except`:

- 1. Quantidade de pessoas — valida que é inteiro ≥ 1
- 2. Nome e CPF de cada hóspede — laço `for` de 1 até `qtd`
- 3. Número de noites — valida que é inteiro ≥ 1
- 4. Tipo de suíte — valida que a chave existe em `SUITES`

Depois chama `calcular_valor_diaria()`, `gerar_codigo()` e `buscar_quarto_disponivel()`. Se tudo estiver ok, monta o dicionário da reserva e salva.

fazer_checkout()

Busca a reserva pelo código, valida que está ativa, e exibe o resumo com todos os gastos. Ao confirmar, marca `ativa=False` e preenche `checkout_dt`. A partir desse momento a reserva passa a ser contabilizada nas estatísticas.

mudar_quarto()

Permite trocar o hóspede de quarto ou até de tipo de suíte. Usa `buscar_quarto_disponivel()` passando o código atual em `cod_excluir` para que o quarto atual não seja considerado 'ocupado'. Recalcula o valor das diárias com o novo tipo de suíte.

consultar_reservas()

Usa uma list comprehension para filtrar as reservas ativas:

```
ativas = [r for r in reservas.values() if r['ativa']]
```

Exibe cada reserva com nome e CPF de todos os hóspedes, quarto, check-in, noites e gastos parciais.

2.6 Bloco de Cardápio

adicionar_item_cardapio()

O usuário escolhe a categoria, digita o nome e o preço. O item é adicionado como tupla à lista daquela categoria: `CARDAPIO[cat].append((nome_item, preco))`. Salva o arquivo após adicionar.

remover_item_cardapio()

Lista os itens numerados da categoria escolhida. O usuário digita o número e o item é removido com `CARDAPIO[cat].pop(idx)`. O índice é validado antes da remoção para evitar `IndexError`.

2.7 Bloco de Gerenciamento de Usuários (Admin)

excluir_usuario(usuario_atual)

Recebe o usuário logado como argumento para impedir que ele se exclua. Monta uma lista de nomes excluindo o próprio usuário:

```
listar_nomes = [n for n in usuarios.keys() if n != usuario_atual['nome']]
```

Após confirmação, remove do dicionário com `del usuarios[nome_alvo]` e salva.

2.8 Bloco de Estatísticas

calcular_totais(dict_reservas, limite)

Recebe o dicionário de reservas e um objeto `datetime` como limite. Percorre todas as reservas e considera apenas as que: (1) estão inativas (`ativa=False`), (2) têm `checkout_dt` preenchido, e (3) o checkout ocorreu após a data limite. Retorna uma tupla com os dois totais.

mostrar_estatisticas(dias, titulo)

Calcula o limite subtraindo `timedelta(days=dias)` da data atual. Chama `calcular_totais()` e exibe os valores com porcentagem e barra visual ASCII.

3. Tabela de Funções

Visão consolidada de todas as funções, seus argumentos, retornos e dificuldade de implementação:

Função	Argumento(s)	Retorno
<code>carregar_dados()</code>	—	Preenche usuarios e reservas
<code>salvar_reservas()</code>	—	Escreve reservas.txt
<code>salvar_usuarios()</code>	—	Escreve usuarios.txt
<code>salvar_cardapio()</code>	—	Escreve cardapio.txt
<code>gerar_codigo(dict_reservas)</code>	dict_reservas	str — código único 4 dígitos
<code>calcular_valor_diaria(suite, qtd, noites)</code>	suite, qtd, noites	float — valor total das diárias
<code>buscar_quarto_disponivel(...)</code>	dict_reservas, suite_id, andar, cod_excluir	int — número do quarto ou None
<code>tem_admin(dict_usuarios)</code>	dict_usuarios	bool
<code>calcular_totais(dict_reservas, limite)</code>	dict_reservas, limite (datetime)	tuple (float, float)
<code>barra(pct, largura)</code>	pct, largura	str — barra ASCII
<code>fazer_login(dict_usuarios)</code>	dict_usuarios	dict do usuário ou None
<code>fazer_checkin()</code>	—	— (modifica reservas)
<code>fazer_checkout()</code>	—	— (modifica reservas)
<code>mudar_quarto()</code>	—	— (modifica reservas)
<code>registrar_pedido()</code>	—	— (modifica reservas)
<code>excluir_usuario(usuario_atual)</code>	usuario_atual	— (modifica usuarios)
<code>mostrar_estatisticas(dias, titulo)</code>	dias (int), titulo (str)	— (apenas exhibe)
<code>setup_inicial()</code>	—	— (cria admin)

4. Funcionamento dos Módulos

5.1 Módulo os

Usado exclusivamente para verificar a existência dos arquivos antes de abri-los:

```
if os.path.exists(ARQUIVO_USUARIOS):
```

Sem essa verificação, tentar abrir um arquivo inexistente no modo leitura ('r') geraria um `FileNotFoundError` e quebraria o programa na primeira execução. O `os.path.exists()` retorna `True` se o arquivo existir no disco e `False` caso contrário, permitindo tratar os dois casos sem `try/except`.

5.2 Módulo random

Usado em uma única função — `gerar_codigo()` — para gerar o número identificador de cada reserva:

```
cod = str(random.randint(1000, 9999))
```

O `random.randint(a, b)` gera um inteiro aleatório entre `a` e `b` inclusive. O resultado é convertido para string com `str()` porque o código é usado como chave do dicionário de reservas, que usa strings.

5.3 Módulo datetime

É o módulo mais utilizado no programa. Duas classes são importadas:

`datetime.now()`

Retorna o momento atual com data e hora. É usado no check-in e checkout para registrar o timestamp:

```
checkin_dt.strftime("%d/%m/%Y %H:%M") # → "23/06/2026 14:30"
checkin_dt.isoformat()                # → '2026-06-23T14:30:00' (para
comparação)
```

O `strftime` formata para exibição ao usuário. O `isoformat` salva um formato preciso que pode ser recarregado com `datetime.fromisoformat()` na leitura do arquivo.

`timedelta`

Representa uma duração de tempo. Usado nas estatísticas para calcular o limite de data:

```
limite = datetime.now() - timedelta(days=30)
```

Subtrair um `timedelta` de um `datetime` retorna outro `datetime`. Isso permite comparar o `checkout_dt` de cada reserva com o limite e saber se está dentro do período de 30 ou 365 dias.

5. Fluxo Geral de Execução

Ao executar o arquivo, o bloco `if __name__ == '__main__':` chama `main()`. O fluxo é:

- 1. `carregar_dados()` — lê os arquivos `.txt` e reconstrói as estruturas em memória.
- 2. `tem_admin()` — se não houver admin, chama `setup_inicial()` para criar o primeiro.
- 3. `tela_inicial()` — exibe login/registro e retorna o usuário logado.
- 4. Dependendo da área do usuário (`administracao` ou `restaurante`), chama o menu correspondente.
- 5. Cada ação que modifica dados chama imediatamente a função de salvar, garantindo que nenhuma alteração seja perdida mesmo se o programa for fechado inesperadamente.

O programa nunca encerra por conta própria — só sai quando o usuário escolhe a opção 0 em `tela_inicial()`, que chama `exit()`.
